GaussBridge: Gaussian World Models for Diverse Start-to-Goal 4D Trajectory Generation

CS771 Project Proposal

Shivam Mittal [smittal39] Amogh Dixit [asdixit3] Aditya Kumar [akumar323]

Problem

Given a few multi-view frames of a scene at a **start** time and a few frames of the same scene at a later **end** time, we want to generate what happened in between: many different trajectories, each as realistic as what could actually have occurred in the real world between those two states, each as a complete 3D world at every time step.

Why it matters

- **Dynamic 3D data is scarce.** Capturing and labeling how a scene evolves (3D motion, depth, point tracks) is expensive; start and end snapshots are cheap.
- **Video generators are not enough.** They give plausible pixels but no consistent 3D world, no exact labels, and no guarantee of reaching the end state.
- **Gap in current work.** Gaussian world models (GWM [3], ManiGaussian [2], 4DGS-WAM [4]) only predict *forward* from the current state; goal-conditioned world models (Act2Goal [10]) work in 2D video; 4D inbetweening (In-2-4D [5]) handles single objects with one output. No method we know of generates **diverse, scene-level** motion conditioned on **both start and end** in a **3D Gaussian** world.
- **Broad use.** The generated worlds can train policies and VLAs, train perception models (tracking, depth, flow), create test scenarios, or serve as a time-indexed 4D world.

Proposed solution

Reconstruct the scene **once** as 3D Gaussians [1], split into a static background and movable parts (objects, vehicles, robot links). A learned **goal-conditioned Gaussian world model** generates how each part moves at every intermediate step, with the start and end states held fixed. The world at any step is the same Gaussians with each part moved to its generated pose, rendered from any camera with exact labels. Trajectories land exactly on the end state, and sampling gives many different ones.

Implementation steps

1. **Capture** multi-view frames of the start and end scenes (simulator cameras, a handheld walk-around, or a camera rig).
2. **Reconstruct** each as 3D Gaussians (`gsplat` [8]), segment them into background and movable parts (SAM 2 [7] or simulator masks), and fuse start and end views to complete hidden sides.
3. **Estimate endpoints:** the start and end pose of every movable part (known in simulation; registration for real scenes).
4. **Train the world model:** a diffusion transformer [6] that outputs each part's pose at every intermediate step, conditioned on part shapes, the obstacle layout, and the fixed start and end poses. All steps are refined jointly, so motion is continuous and plans ahead.
5. **Sample** many trajectories per start/end pair, steer them away from collisions, filter invalid ones, and keep the most diverse.
6. **Render** each step from any camera: RGB, depth, optical flow, 3D point tracks and masks.
7. **Export** the 4D worlds, or package them as training data for downstream models.

Evaluation

On held-out simulated scenes (with ground-truth motion and video) and real captured scenes:

- **Endpoint error** (cm, degrees): does every trajectory reach the end state?
- **Collision rate and smoothness:** is the motion physically valid and continuous?
- **Coverage and diversity vs. reference paths:** is it realistic, and does it capture the different ways the motion could happen?
- **PSNR / LPIPS [9] vs. ground-truth video:** do the rendered worlds look real?
- **Downstream gain:** does a model trained with our data (a policy or a point tracker) beat the same model without it?

Baselines: (1) *Straight line*: each object moves directly from start to end at constant speed. (2) *Forward-only*: our model without the end state, as current Gaussian world models work. (3) *Step-by-step*: our model generating a few steps at a time, each chunk built on the previous one.

Success criterion: exact endpoints, realistic trajectories, fewer collisions and higher coverage than the baselines, and a measurable gain on at least one downstream task.

References

[1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[2] Guanxing Lu, Shiyi Zhang, Ziwei Wang, Changliu Liu, Jiwen Lu, and Yansong Tang. ManiGaussian: Dynamic Gaussian splatting for multi-task robotic manipulation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.

[3] Guanxing Lu, Baoxiong Jia, Puhao Li, Yixin Chen, Ziwei Wang, Yansong Tang, and Siyuan Huang. GWM: Towards scalable Gaussian world models for robotic manipulation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

[4] Yueen Ma, Zenglin Xu, and Irwin King. 4DGS-WAM: Bridging past and future with an object-centric world action model based on 4D Gaussian splatting. *arXiv preprint arXiv:2608.25956*, 2026.

[5] Sauradip Nag, Daniel Cohen-Or, Hao Zhang, and Ali Mahdavi-Amiri. In-2-4D: Inbetweening from two single-view images to 4D generation. *arXiv preprint arXiv:2504.08366*, 2025.

[6] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[7] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations (ICLR)*, 2025.

[8] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for Gaussian splatting. *arXiv preprint arXiv:2409.06765*, 2024.

[9] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[10] Pengfei Zhou, Liliang Chen, Shengcong Chen, Di Chen, Wenzhi Zhao, Rongjun Jin, Guanghui Ren, and Jianlan Luo. Act2Goal: From world model to general goal-conditioned policy. *arXiv preprint arXiv:2512.23541*, 2025.